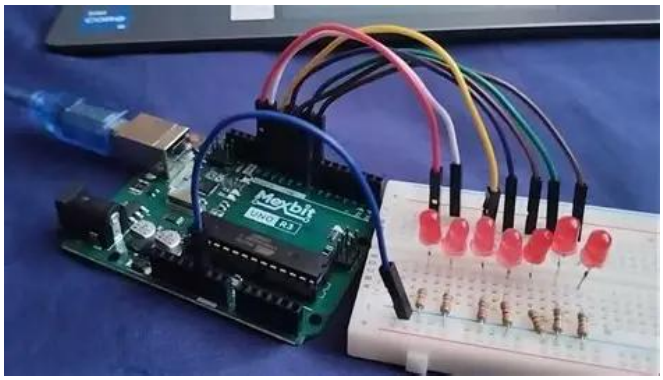
UNIVERSIDAD AUTONOMA DE TAMAULIPAS.

UNIDAD ACADÉMICA MULTIDISCIPLINARIA DEL MANTE.

Arquitectura de Equipos Digitales.

Programación de Interfaces de Puertos.

PROYECTO DEL SEGUNDO PARCIAL AED Y PIP



Integrantes:

- Arizabalo Sánchez Ivonne Michelle.
- Garza Martínez Nelson.
- Hernández Cardenas Perla Elizabeth.
- Hernández Castro Abigail.
- Ibarra Chalí Ángel Osiris.
- Jiménez Martínez Luis Gerardo.
- Sánchez González Cecilia Moncerrat.

Docente: López Piña Daniel.

23 de abril del 2026.

MEDIDOR DE TEMPERATURA CON PANTALLA INTEGRADA

Introducción:

En este proyecto se desarrolló un circuito sencillo que permite conocer la temperatura, la humedad y detectar si hay algún objeto cercano. Para esto se utilizó un Arduino junto con sensores y una pantalla donde se muestran los datos en tiempo real. La idea principal es poder observar las condiciones del entorno de manera rápida y clara. Este tipo de sistemas puede ser útil tanto en el hogar, entornos, negocios, bodegas como en otros espacios, ya que ayuda a estar al pendiente del ambiente y a reaccionar si algo cambia.

Planteamiento del problema:

Un negocio necesita una alarma que se base en la temperatura de una zona de refrigerados que hasta que suba a una cierta temperatura emita una alarma y a su vez se pueda mostrar en pantalla la temperatura actual y la humedad y así se evitan perdidas de productos del negocio.

Objetivo:

El objetivo de este proyecto es crear un sistema que sea capaz de medir la temperatura, la humedad y la distancia, mostrar esta información en una pantalla y encender una luz y una alerta sonora como aviso cuando los valores sean altos.

Materiales:

- Arduino UNO
- Cable USB
- Sensor DHT11
- LCD 16x2 con adaptador 12C
- Módulo HC-SR04
- Cables
- LED
- Laptop Arduino IDE
- Librería Liquid Crystal _12C
- Protoboard

Justificación de arquitectura utilizada:

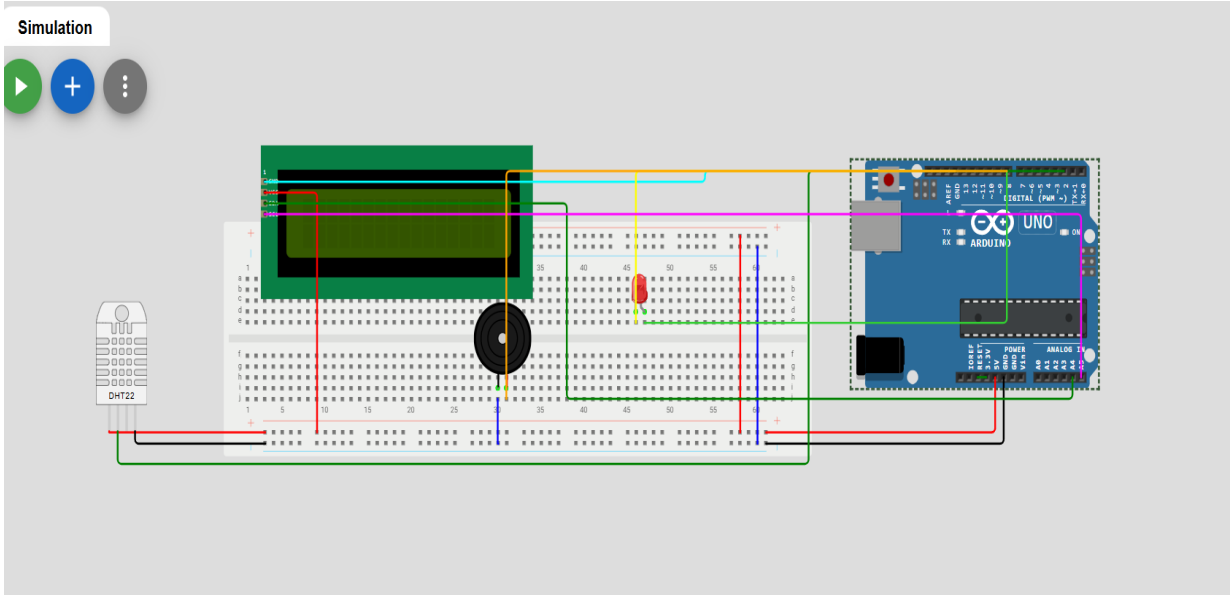
La arquitectura es ideal para un sistema de control de seguridad industrial o doméstico. El Arduino Uno centraliza los datos del DHT11 y el HC-SR04, decide si los valores son críticos para activar el Buzzer/LED y envía la información procesada de forma eficiente a través del bus I2C hacia la pantalla LCD. Es una solución autónoma que no depende de redes externas para funcionar.

Protocolos de comunicación:

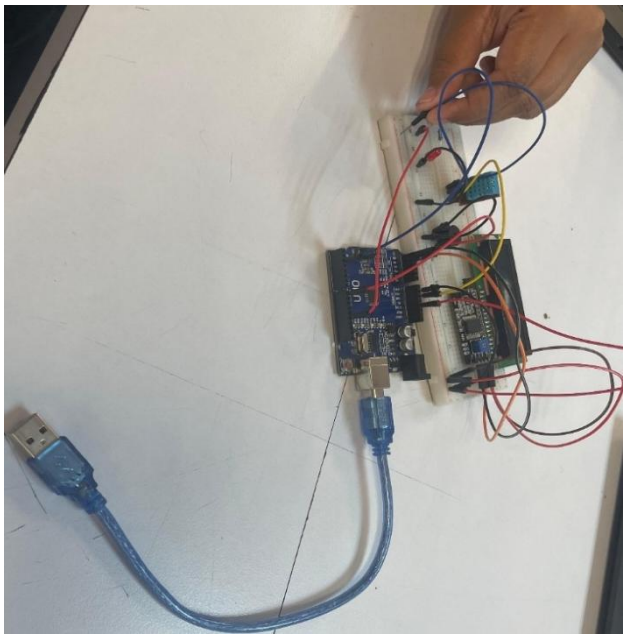
- Protocolo “Single-Bus” (DHT) – Protocolo propietario serial de un solo hilo.

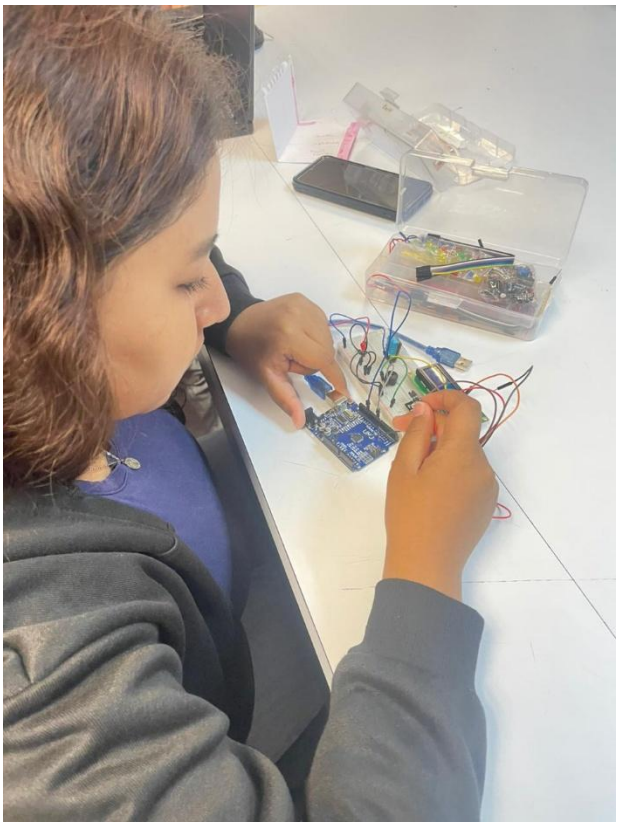
Desarrollo:

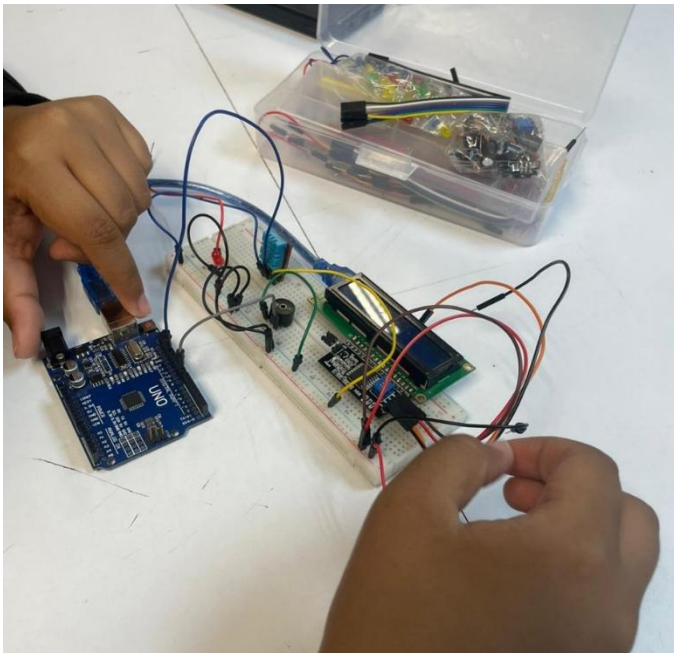
Se realizó el diagrama de conexión del circuito en Wokwi para planificar el armado. Lo cual permitió visualizar de forma clara cómo debían ir conectados todos los componentes antes de armarlo físicamente.

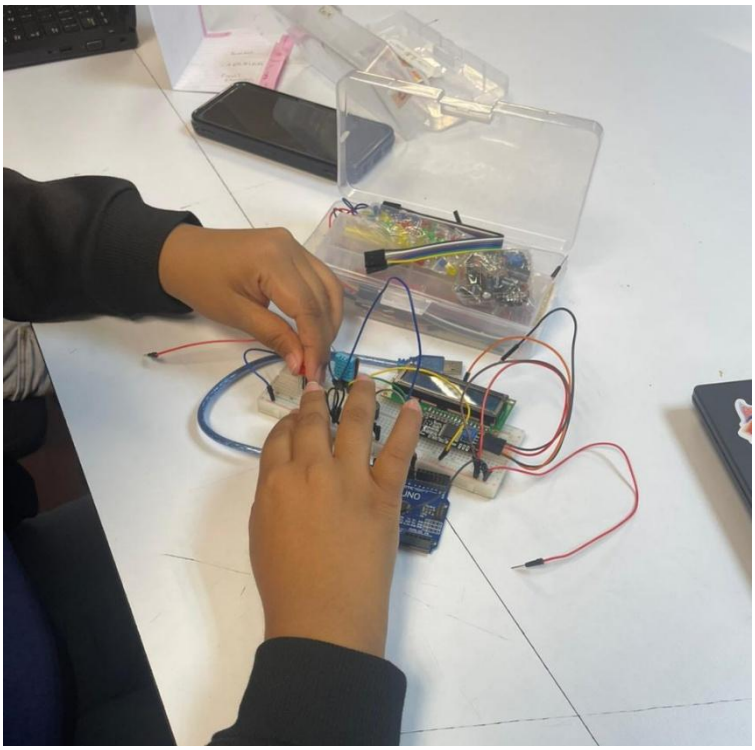


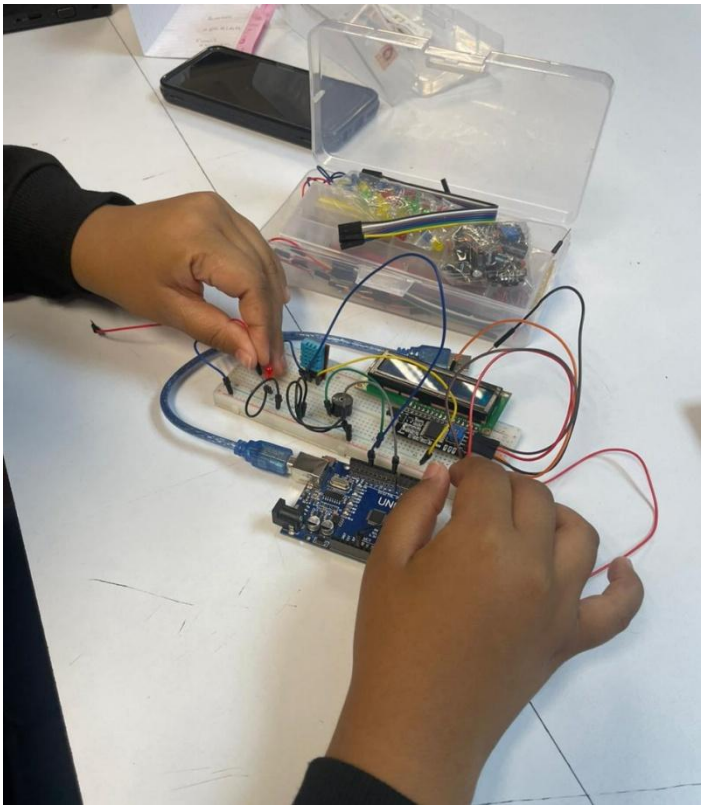
Se realizó el armado del circuito tomando como guía el esquema de conexión previamente diseñado en Wokwi. Esto permitió conectar correctamente cada componente y asegurar un funcionamiento adecuado del sistema.











Posteriormente, se verificó el código del programa en Arduino IDE para asegurarse de que funcionara correctamente antes de cargarlo en el Arduino. Esto permitió detectar posibles errores y garantizar una correcta conexión y ejecución del sistema.

Código:

```
#include <Wire.h>

#include <LiquidCrystal_I2C.h>

#include <DHT.h>

// -------- CONFIGURACIÓN --------

#define DHTPIN 2

#define DHTTYPE DHT11


DHT dht(DHTPIN, DHTTYPE);

LiquidCrystal_I2C lcd(0x27, 16, 2);


const int LED = 13;

const int BUZZ = 8;

const int ALERTA_TEMP = 25; // °C


// -------- SETUP --------
```

```
void setup() {
  pinMode(LED, OUTPUT);
  pinMode(BUZZ, OUTPUT);

  Serial.begin(9600);
  dht.begin();

  lcd.init();
  lcd.backlight();
  lcd.print("Iniciando...");
  delay(1500);
}

// -------- LOOP --------
void loop() {
  float temp = dht.readTemperature();
  float hum = dht.readHumidity();

  lcd.clear();

  // Validar lectura
```

```
if (isnan(temp) || isnan(hum)) {
  lcd.setCursor(0, 0);
  lcd.print("Error sensor");

  digitalWrite(LED, LOW);
  digitalWrite(BUZZ, LOW);

  Serial.println("Error sensor");
  delay(1000);
  return;
}
// Mostrar en LCD
lcd.setCursor(0, 0);
lcd.print("Temp: ");
lcd.print(temp);
lcd.print(" C");

lcd.setCursor(0, 1);
lcd.print("Hum: ");
lcd.print(hum);
```

```
lcd.print("
%");

  // Monitor serial
  Serial.print("Temp: ");
  Serial.print(temp);
  Serial.print(" C  Hum: ");
  Serial.print(hum);
  Serial.println(" %");

  // Alerta
  bool alerta = (temp > ALERTA_TEMP);
  digitalWrite(LED, alerta);

  if (alerta) {
    tone(BUZZ, 1000); // sonido buzzer
  } else {
    noTone(BUZZ);
  }
  delay(1000);
}
```

Después de realizar pruebas en la etapa tester, se analizaron los resultados obtenidos del sistema. Se registraron los siguientes valores:

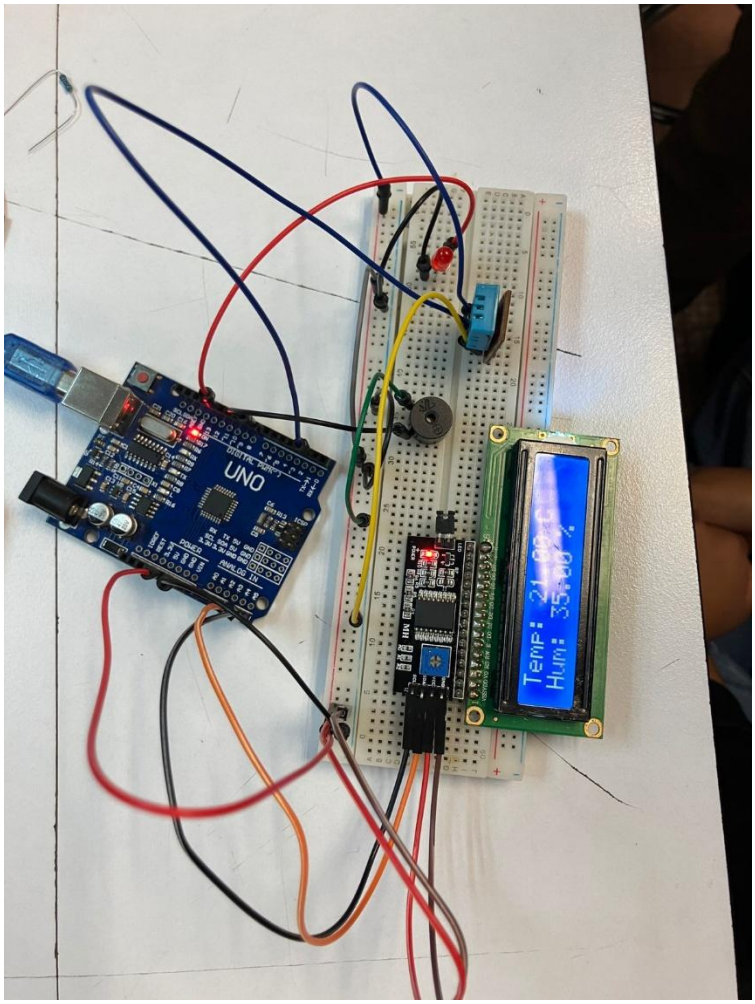
- **Temperatura:** entre 25 °C y 26 °C
- **Humedad:** 35%

Se observó que la temperatura presentó una ligera variación entre 25 y 26 grados Celsius, mientras que la humedad se mantuvo constante en un 35%.

En cuanto al código, una característica importante es la validación de los datos de temperatura y humedad, utilizando una condición para detectar errores en la lectura del sensor. En caso de falla, el sistema muestra un mensaje de error en la pantalla LCD.

Además, se notó que el código está dividido en varias partes, lo que lo hace un poco más largo. Como mejora, se podría simplificar para hacerlo más claro y fácil de entender.

Resultado:



Después de realizar pruebas en la etapa tester y comprobar que todo funcionaba correctamente, se llevó a cabo la conexión final del circuito. El sistema operó de manera adecuada, mostrando en la pantalla los valores de temperatura y humedad en tiempo real. Además, componentes como el LED respondieron según lo programado, lo que confirma que tanto el circuito como el código fueron correctos.

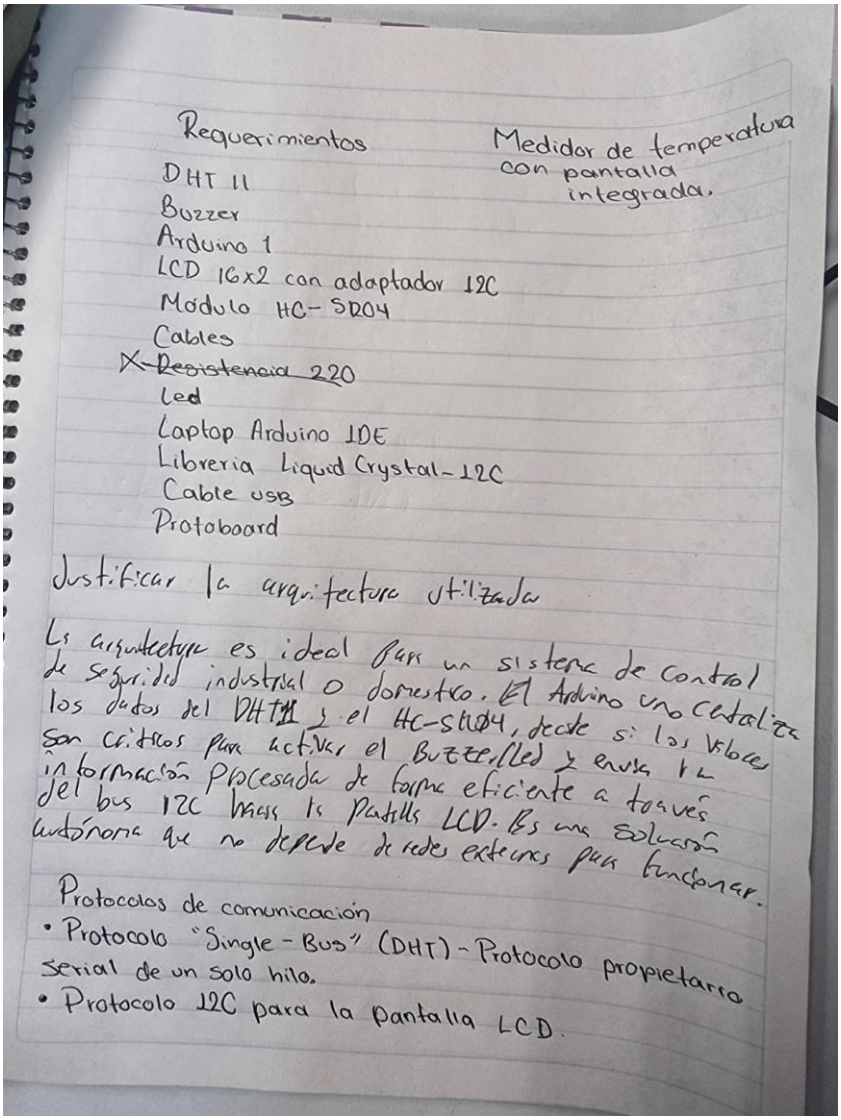
Evidencias:

Etapa 1: Analistas de Requisitos

Estudiantes Encargados:

- Hernández Cardenas Perla Elizabeth.
- Jiménez Martínez Luis Gerardo.

Analizan los requisitos del sistema, identifican componentes necesarios, justifican la arquitectura utilizada, protocolos de comunicación y elaboran la lista de materiales. Presentan propuesta al equipo.



Requerimientos

Medidor de temperatura con pantalla integrada.

DHT 11
Buzzer
Arduino 1
LCD 16x2 con adaptador I2C
Modulo HC-SR04
Cables
~~X-Resistencia 220~~
Led
Laptop Arduino IDE
Libreria Liquid Crystal-I2C
Cable usb
Protoboard

Justificar la arquitectura utilizada

La arquitectura es ideal para un sistema de control de seguridad industrial o domestico. El Arduino uno centaliza los datos del DHT11 y el HC-SR04, desde si los valores son criticos para activar el Buzzer/Led y envia la información procesada de forma eficiente a través del bus I2C hacia la Pantalla LCD. Es una solución autónoma que no depende de redes externas para funcionar.

Protocolos de comunicación

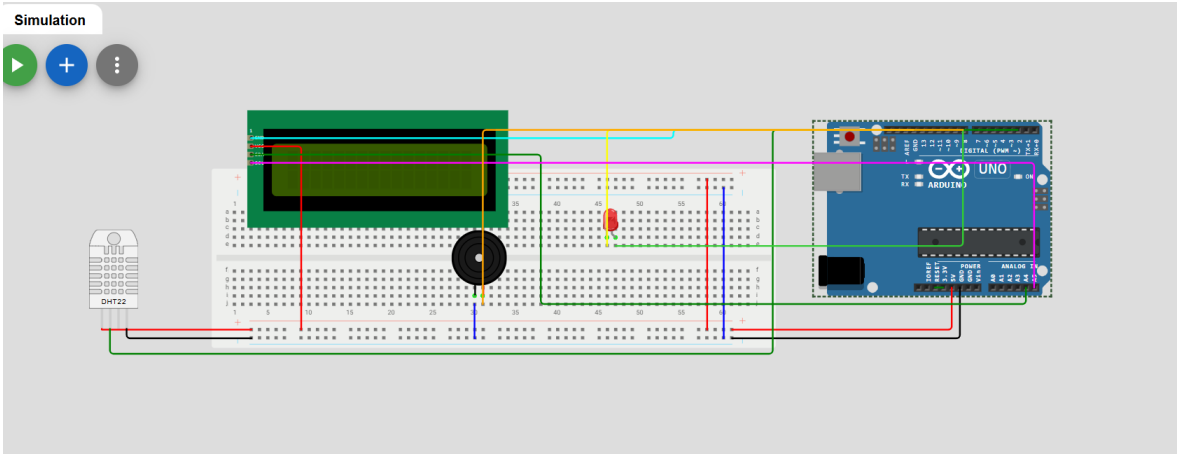
- Protocolo "Single-Bus" (DHT) - Protocolo propietario serial de un solo hilo.
- Protocolo I2C para la Pantalla LCD.

Etapa 2: Diseñador de Circuito

Estudiante Encargado:

- Arizabalo Sánchez Ivonne Michelle

Realiza el esquema de conexión (diagrama de conexiones y/o esquemático). Valida el diseño antes de entregar al montador.

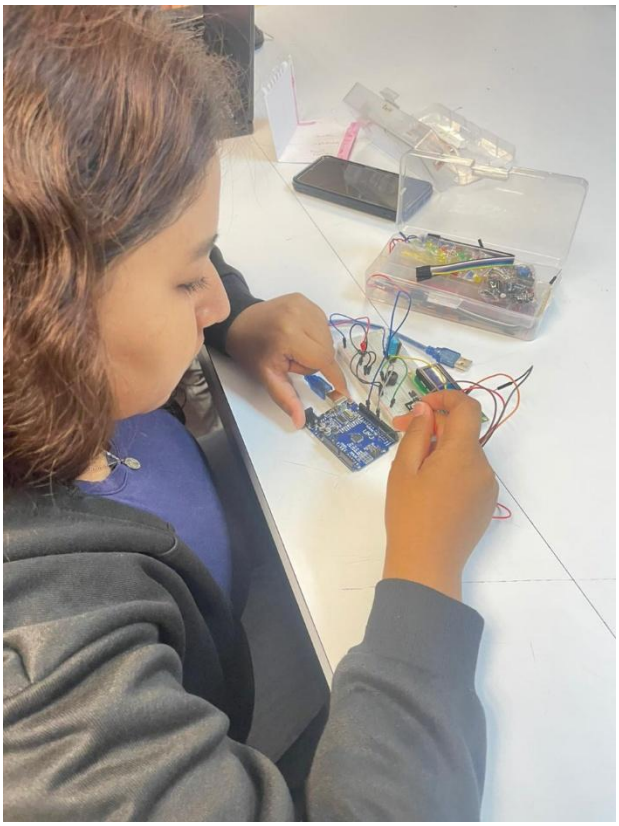


Etapa 3: Montador de Hardware

Estudiante Encargado:

- Sánchez González Cecilia Moncerrat.

Realiza las conexiones físicas en Protoboard/PCB siguiendo el esquema. No modifica el diseño original.

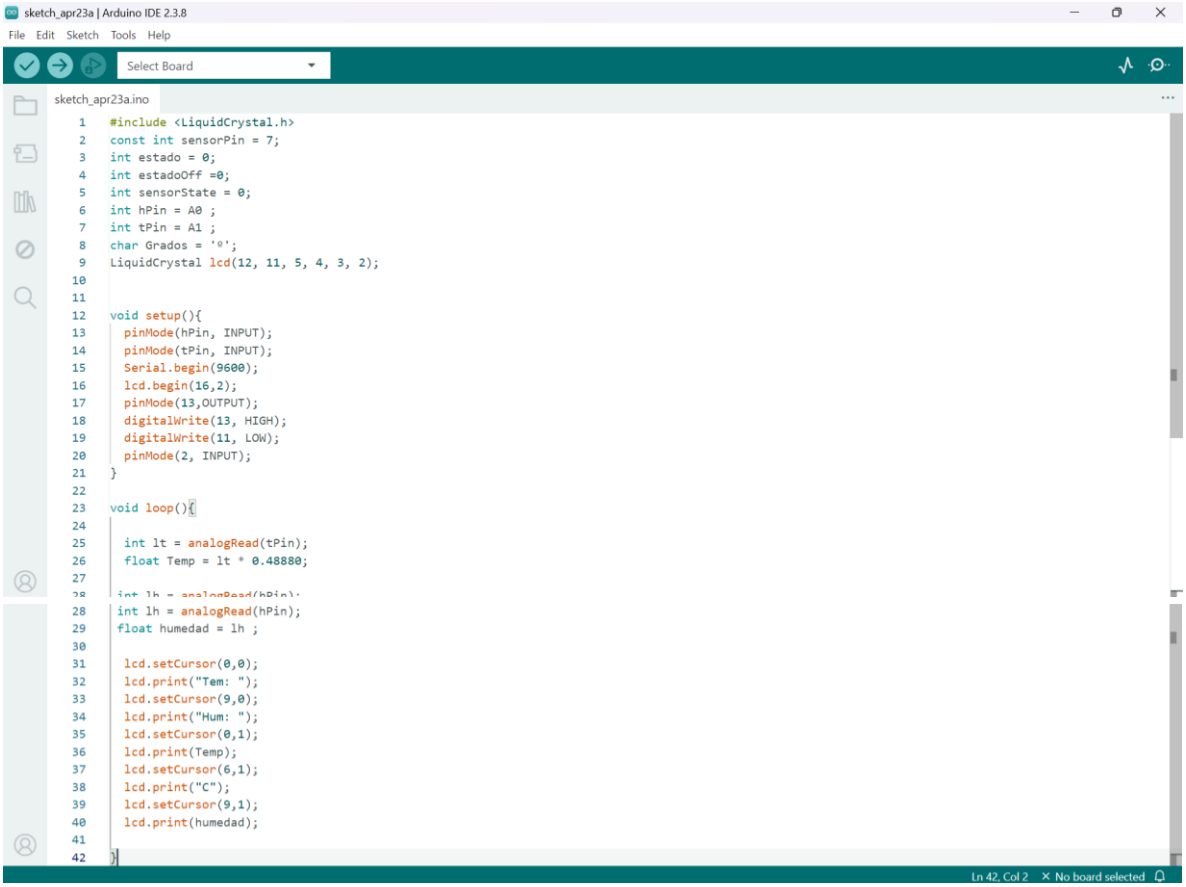


Etapa 4: Programador

Estudiante Encargado:

- Garza Martínez Nelson.

Codifica el firmware en Arduino IDE. Sólo puede comenzar cuando las conexiones estén verificadas.




```
#include <LiquidCrystal.h>
const int sensorPin = 7;
int estado = 0;
int estadoOff =0;
int sensorState = 0;
int hPin = A0 ;
int tPin = A1 ;
char Grados = '°';
LiquidCrystal lcd(12, 11, 5, 4, 3, 2);


void setup(){
  pinMode(hPin, INPUT);
  pinMode(tPin, INPUT);
  Serial.begin(9600);
  lcd.begin(16,2);
  pinMode(13,OUTPUT);
  digitalWrite(13, HIGH);
  digitalWrite(11, LOW);
  pinMode(2, INPUT);
}

void loop(){

  int lt = analogRead(tPin);
  float Temp = lt * 0.48880;

  int lh = analogRead(hPin);
  float humedad = lh ;

  lcd.setCursor(0,0);
  lcd.print("Tem: ");
  lcd.setCursor(9,0);
  lcd.print("Hum: ");
  lcd.setCursor(0,1);
  lcd.print(Temp);
  lcd.setCursor(6,1);
  lcd.print("C");
  lcd.setCursor(9,1);
  lcd.print(humedad);

}
```

Etapa 5: Tester/QA

Estudiante Encargado:

- Ibarra Chalí Ángel Osiris.

Realiza pruebas funcionales del sistema. Documenta resultados de cada prueba y confirma la completitud.

Temp: 22.00 C -21.00 C
Hum: 35.00%

```
const int LeD=13;
const int Buzz=8;
const int Alerta_temp=30;

// Validar lectura
if (isnan(temp) || isnan(hum)){
lcd.setCursor(0,0);
lcd.Print("Error sensor");

//Mostrar en LCD
lcd.setCursor(0,0);
lcd.print("Temp: ");
lcd.print(temp);
lcd.print("C");

lcd.setCursor(0,1);
lcd.print("Hum: ");
lcd.print(hum);
lcd.print("%");
```

Las caracteristicas mas relevantes que encontre que la "temp:" tenia una variacion entre 22 y 21 grados celsios mientras que habia un "hum:" que se mantuvo constante en 35%

en mi opinion

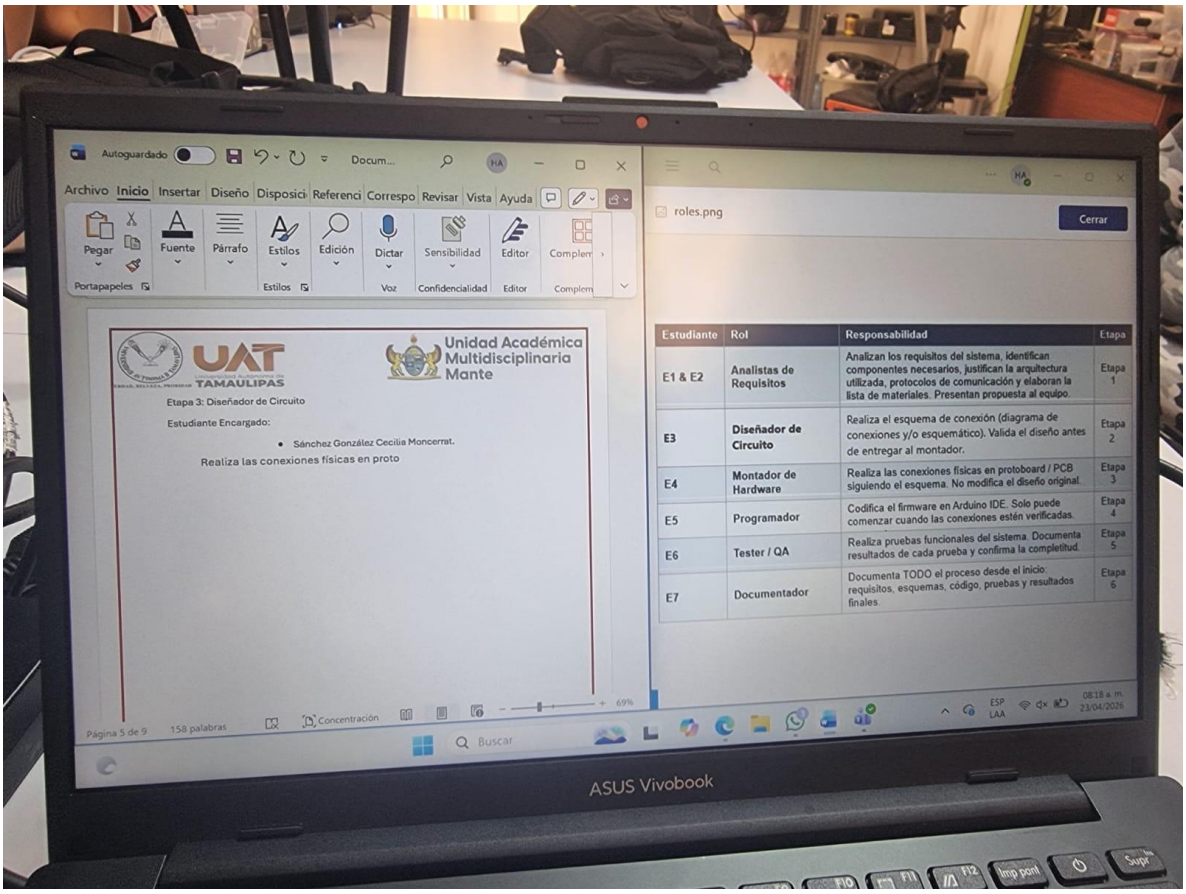
Una caracteristica que note fue el codigo necesitaba validar campos "temp" y "hum". otra caracteristica que note es que el codigo es muy complicado y se divide en varias partes

Etapa 6: Documentador

Estudiante Encargado:

- Hernández Castro Abigail.

Documenta TODO el proceso desde el inicio: Requisitos, esquemas, código, pruebas y resultados finales.



Conclusión:

Al finalizar el proyecto, se obtuvo un sistema funcional que permite monitorear el entorno de forma simple. Se pudo comprobar que es posible integrar diferentes componentes para obtener información útil y mostrarla de manera clara. Además, este proyecto demuestra que no se necesitan herramientas complicadas para crear soluciones que pueden ayudar en la vida diaria. Es una buena base para desarrollar sistemas más completos en el futuro.